DOEN PA

รายงานระบบ และสถาปัตยกรรม

สมุดพาสปอร์ตเดินป่าดิจิทัลสำหรับนักเดินป่าชาวไทย — เครื่องมือ การเชื่อมต่อ โครงสร้างไฟล์ ข้อมูล สถาปัตยกรรม data flow workflow สถานการณ์ใช้งาน และข้อแนะนำ

จัดทำวันที่ 30 ตุลาคม 2569 (30 Oct 2026)

Next.js 16.4.2 · React 19 · Supabase · Vercel

https://doen-pa.vercel.app

สารบัญ

1. ภาพรวมโปรเจค
2. เครื่องมือและเทคโนโลยีที่ใช้ (Frontend · Backend · Database · Infrastructure)
3. สถาปัตยกรรมระบบ (Architecture)
4. การเชื่อมต่อและการรับส่งข้อมูล ระหว่างไคลเอนต์ เซิร์ฟเวอร์ และอินเทอร์เน็ต
5. โครงสร้างไฟล์และหน้าที่ของแต่ละไฟล์
6. โมเดลข้อมูลและความสัมพันธ์ของข้อมูล
7. Data Flow
8. Workflow (การพัฒนา การนำเข้าข้อมูล CI/CD)
9. สถานการณ์ใช้งาน (Scenarios)
10. ความปลอดภัยและความเป็นส่วนตัว
11. สิ่งที่ควรรู้ ข้อจำกัด และความเสี่ยง
12. คำแนะนำและสิ่งที่ควรทำต่อ
13. ภาคผนวก: คำสั่งที่ใช้บ่อยและเอกสารอ้างอิง

> **วิธีอ่านรายงานนี้** เนื้อหาสรุปจากโค้ดและเอกสารใน repo ณ วันที่จัดทำ ส่วนใดที่เป็นการประมาณหรือยังไม่ได้ตรวจกับระบบจริงจะระบุไว้ในข้อความ ค่าตั้งของ Supabase dashboard (เช่น rate limit, เทมเพลตอีเมล) อ้างตามที่ผู้ดูแลแจ้ง ไม่ได้อ่านจากโค้ด

1. ภาพรวมโปรเจค

Doen Pa (เดิ่นป่า) คือ Digital Hiking Passport ส่วนตัวสำหรับนักเดินป่าชาวไทย แนวคิดหลักคือ **"บันทึกการเดินทาง ไม่ใช่วางแผนการเดินทาง"** ผู้ใช้ค้นหาสถานที่ → "ประทับตรา" การเยือนด้วยการสร้าง travel record (วันที่ รูป ข้อความ คะแนนส่วนตัว) → ดูความทรงจำใน Passport, หน้ารายละเอียดสถานที่, แผนที่, สถิติ และโปรไฟล์

156	123	7	63
อุทยานแห่งชาติใน catalog	แห่งมีรูปถ่ายลิขสิทธิ์เปิด	หน้าหลัก (ค้นหา แผนที่ พาสปอร์ต สถิติ โปรไฟล์ ...)	unit/DB test (+ e2e Playwright 14 สเปค)

สิ่งที่อยู่ในขอบเขต (MVP)

- ค้นหา/กรองสถานที่ (ประเภท ภูเขา น้ำตก ถ้ำ หมู่เกาะ · เคยไปแล้ว/ยังไม่เคย)
- หน้ารายละเอียดสถานที่ และการประทับตราสร้างบันทึกการเดินทาง (สูงสุด 5 รูปต่อบันทึก)
- Passport (รายการบันทึกทั้งหมด) · แผนที่เฉพาะที่เคยไป · สถิติรายจังหวัด/ภาค/ประเภท/อุทยาน · โปรไฟล์
- บัญชีผู้ใช้ด้วยอีเมล/รหัสผ่าน (สมัคร ยืนยันอีเมล เข้าสู่ระบบ กู้รหัสผ่าน ออกจากระบบ)

สิ่งที่อยู่นอกขอบเขตจนกว่าจะอนุมัติ

ระบบ badge, social, โปรไฟล์/รีวิว/คะแนนสาธารณะ, จอง/ชำระเงิน, วางแผนเส้นทาง, GPS สด, พยากรณ์อากาศ, ระบบแนะนำ, activity feed, ข้อความ — คะแนนดาวของผู้ใช้เป็นข้อมูลส่วนตัวเท่านั้น ไม่ถูกรวมเป็นคะแนนสาธารณะ

สถานะปัจจุบัน

- รันจริงบน Vercel ที่ `https://doen-pa.vercel.app` ใช้ Supabase (Auth, Postgres, Storage) เป็น backend ทั้งหมด
- ดีไซน์ปัจจุบันคือ "Contour & Stamp" (เขียวป่า พื้นหลังเส้นชั้นความสูง ตราประทับหมึก กระจก Liquid Glass)
- เลื่อนไว้: Google login, หน้า badge, นโยบายรูปสถานที่ถาวรระยะยาว

2. เครื่องมือและเทคโนโลยีที่ใช้

2.1 Frontend (ฝั่งแสดงผล)

เครื่องมือ	บทบาทในโปรเจค
Next.js 16.4.2 (App Router)	เฟรมเวิร์กหลัก ทั้งหน้าเว็บ (Server/Client Components) และ Route Handlers เวอร์ชันนี้มีการเปลี่ยนแปลงจากที่เคยรู้จัก เช่น ใช้ `proxy.ts` แทน middleware เดิม และ build ด้วย Webpack (`next build --webpack`)
React 19	UI และ hooks; ใช้ `useSyncExternalStore` ผูก state บัญชีกับหน้าจอ
TypeScript (strict)	ความถูกต้องของชนิดข้อมูล; ชนิดหลักอยู่ที่ `src/types/index.ts`
Tailwind CSS 4	สไตล์ทั้งหมด; token ดีไซน์อยู่ที่ `app/globals.css` และ `tailwind.config.ts`
Leaflet 1.9	แผนที่ (เรียกใช้ตรง ไม่ผ่าน wrapper) โหลดแบบไม่ใช้ SSR; กระเบื้องแผนที่จาก Stadia Maps (Stamen Terrain)
Lucide React	ไอคอน
ฟอนต์ Kanit, Plus Jakarta Sans	บรรจุไว้ใน `public/fonts/` ไม่ดึงจาก Google ตอน build

ไม่มี Zustand หรือ store ภายนอก: state มาจาก React hooks, พารามิเตอร์ใน URL (เช่น `q`, `type`, `visited` ของหน้าค้นหา) และ snapshot ของบัญชีที่ยืนยันแล้วจาก Supabase

2.2 Backend (ฝั่งเซิร์ฟเวอร์)

เครื่องมือ	บทบาท
Next.js Server Components	โหลด catalog สถานที่ (`getPlaceCatalog`) และตรวจตัวตนก่อน render ข้อมูลส่วนตัว
Route Handlers	`/api/account` (อ่านข้อมูลบัญชี), `/api/photos/[id]` (ส่งรูปส่วนตัว), `/auth/[action]` (เข้าสู่ระบบ ออก สมัคร ฯลฯ), `/auth/callback`, `/auth/confirm`
proxy.ts	รันก่อนทุกหน้า (ยกเว้นไฟล์ static) เพื่อรีเฟรช session cookie ของ Supabase ด้วย `getClaims`
@supabase/ssr + **supabase-js**	ไลบรารีเชื่อม Supabase ทั้งฝั่งเซิร์ฟเวอร์ (อ่าน/เขียน cookie) และเบราว์เซอร์
sharp	ย่อรูปและหมุนตามแนวภาพเป็น WebP ใน `/api/photos/[id]` (ขนาด 256/640/1280) และใช้ทำรูปอุทยานตอนเตรียมข้อมูล

2.3 Database และ Storage (Supabase)

ส่วนประกอบ	บทบาท
Supabase Auth	อีเมล/รหัสผ่าน ยืนยันอีเมลผ่านลิงก์ TokenHash กู้รหัสผ่าน; ส่งอีเมลผ่าน Gmail SMTP (บัญชีผู้ส่งเฉพาะ + App Password เก็บใน Supabase เท่านั้น)

PostgreSQL + RLS	ตาราง `places` (catalog สาธารณะอ่านอย่างเดียว), `profiles`, `travel_records`, `photos` (ส่วนตัวต่อเจ้าของด้วย Row Level Security) และฟังก์ชัน RPC เช่น `save_travel_record`, `save_account_profile`, `delete_my_account`
Supabase Storage	bucket ส่วนตัว `travel-photos` (รูปบันทึก ≤5 MB) และ `profile-photos` (อวตาร์/ปก ≤1 MB) เข้าถึงได้เฉพาะเจ้าของ

2.4 Infrastructure และบริการภายนอก

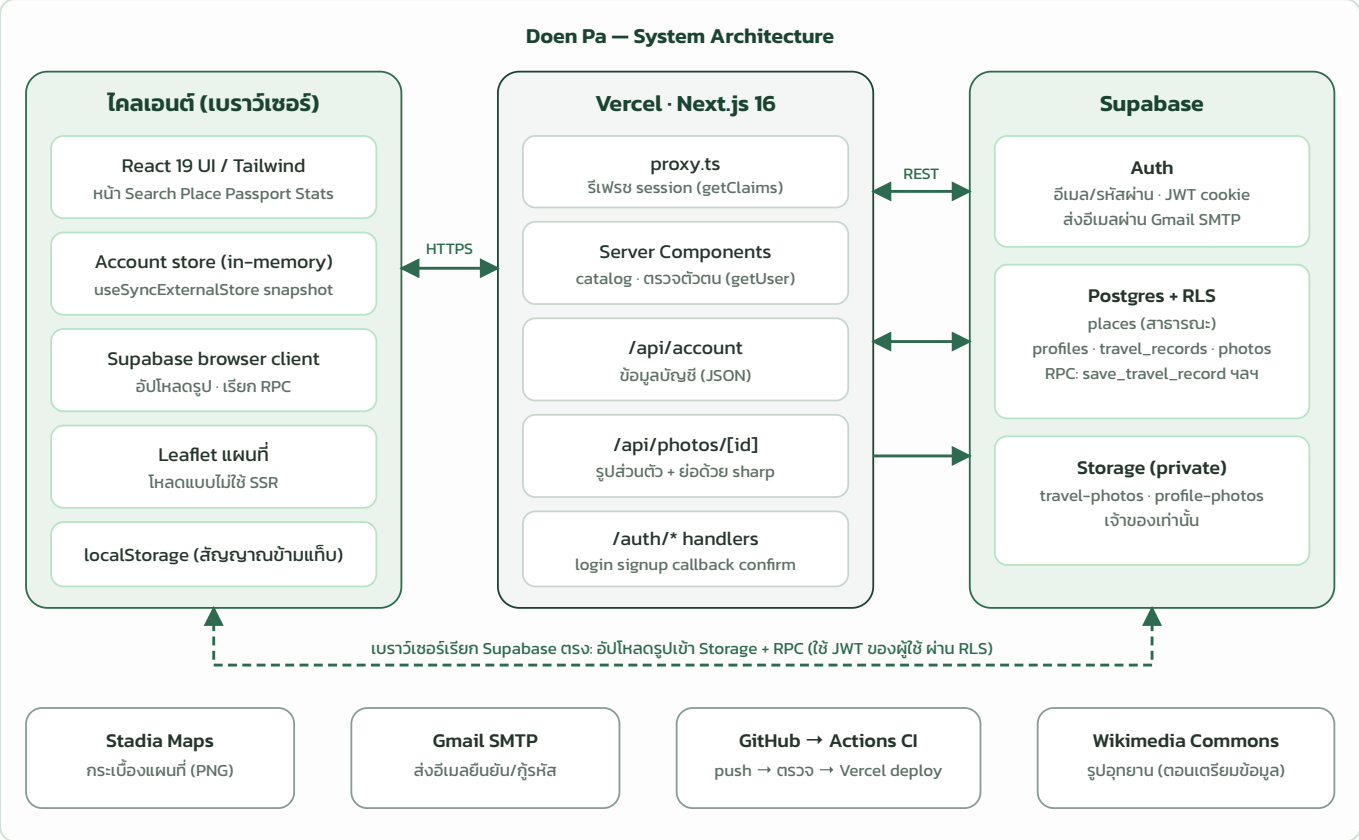
บริการ	บทบาท
Vercel	โฮสต์แอป (deploy จาก `main`); โดเมน doen-pa.vercel.app (ไม่มีโดเมนซื้อเอง)
GitHub + GitHub Actions	เก็บโค้ด; CI รัน secret scan, dependency audit, tsc, lint, test, build ทุกครั้งที่ push
Stadia Maps	กระเบื้องแผนที่ภูมิประเทศ ยืนยันด้วยโดเมน (ไม่มี API key) — ดูข้อควรรู้เรื่องแผน Free ในบทที่ 11
Gmail SMTP	ส่งอีเมลยืนยัน/กู้รหัสผ่าน ผ่านการตั้งค่า Custom SMTP ของ Supabase
Wikimedia Commons	แหล่งรูปอุทยาน (ใช้ตอนเตรียมข้อมูลเท่านั้น ไม่ได้เรียกขณะผู้ใช้ใช้งาน)

2.5 เครื่องมือพัฒนาและทดสอบ

pnpm (ตัวจัดการแพ็กเกจ, Node 22+) · **ESLint** · **PGlite** (Postgres จำลองในหน่วยความจำสำหรับทดสอบ migration/seed/RLS โดยไม่ต้องใช้ Docker) · **Playwright** (e2e บนเดสก์ท็อปและมือถือ) · **Supabase CLI** (push migration, รัน SQL) · **Python 3** (สคริปต์นำเข้า catalog จาก CSV)

3. สถาปัตยกรรมระบบ (Architecture)

ระบบเป็นแอป Next.js ตัวเดียวบน Vercel ที่คุยกับ Supabase เป็นแหล่งข้อมูลเดียว ไคลเอนต์เรียก Supabase ได้โดยตรงสำหรับการเขียนที่ต้องมีธุรกรรมและการอัปโหลดรูป ส่วนการอ่านข้อมูลส่วนตัวผ่านเซิร์ฟเวอร์ของเราเพื่อให้ตรวจตัวตนก่อนทุกครั้ง



รูปที่ 1 สถาปัตยกรรมภาพรวม เส้นทึบ = ผ่านเซิร์ฟเวอร์ Next.js เส้นประ = เบราว์เซอร์เรียก Supabase โดยตรง

หลักการออกแบบสำคัญ

- **Server Components เป็นค่าเริ่มต้น** ใช้ Client Components เฉพาะเรื่องที่ต้องใช้ API เบราว์เซอร์ (Leaflet ฟอร์ม ตัวกรอง state บัญชี)
- **ความจริงอยู่ที่ฐานข้อมูล** ไม่มี catalog สำรองในแอป ถ้าโหลด catalog ไม่ได้ จะแสดงหน้า "โหลดข้อมูลไม่สำเร็จ" แทนที่จะแสดงสถิติว่างเปล่าที่ผิดความจริง
- **ตรวจสิทธิ์สองชั้น** เซิร์ฟเวอร์ตรวจตัวตนก่อน render ข้อมูลส่วนตัว และ RLS ในฐานข้อมูลบังคับสิทธิ์อีกชั้นแม้โค้ดผิดพลาด
- **เขียนข้อมูลแบบ atomic** บันทึกการเดินทางและรูปที่เกี่ยวข้องถูกยืนยันเข้าฐานข้อมูลในฟังก์ชัน RPC เดียว

4. การเชื่อมต่อและการรับส่งข้อมูล

ตารางนี้แสดงว่าใครคุยกับใคร ส่งอะไร ด้วยการยืนยันตัวตนแบบไหน และแคชอย่างไร (ไคลเอนต์ ↔ เซิร์ฟเวอร์ ↔ อินเทอร์เน็ต) การสื่อสารทั้งหมดเป็น HTTPS

4.1 ไคลเอนต์ ↔ เซิร์ฟเวอร์ Next.js (Vercel)

ช่องทาง	เมธอด	ข้อมูลที่ส่ง → ได้รับ	ยืนยันตัวตน/แคช
หน้าเว็บ `/search, /places/[id], /passport …`	GET	ส่ง cookie session + query (`q, type, visited, from, photo`) → ได้ HTML/RSC ที่ render แล้ว (catalog 156 แห่งฝังมากับหน้า Search)	cookie Supabase (SameSite=Lax) · ข้อมูลส่วนตัวไม่แคชร่วม
`/api/account`	GET	ส่ง cookie → ได้ JSON: โปรไฟล์ ข้อมูลบันทึกการเดินทาง และ metadata รูป ของผู้ใช้คนนั้น (ดึงซ้ำเมื่อโฟกัสหน้าต่าง/แท็บอื่นเปลี่ยนข้อมูล)	`getUser()` ยืนยันจริง · `no-store`
`/api/photos/[id]?owner=&w=`	GET	ส่ง id รูป + owner + ความกว้าง (256/640/1280) → ได้ไฟล์รูป (WebP ย่อแล้ว หรือต้นฉบับ)	`getClaims()` + RLS · `private, max-age=1 ปี, immutable` (เบราว์เซอร์เดียวของเจ้าของ)
`/auth/login, signup, resend, forgot-password, logout, complete-profile-setup, delete-account, …`	POST (JSON)	ส่งอีเมล/รหัสผ่าน หรือคำสั่ง → ได้ JSON ผลลัพธ์ + `next` ปลายทาง; ตั้ง/ล้าง cookie session	รับเฉพาะคำขอ same-origin แบบ JSON · `private, no-store` · ส่ง `Clear-Site-Data: "cache"` เมื่อ login/logout/ลบบัญชี
`/auth/confirm`	GET	ลิงก์ในอีเมล: `token_hash` + `type` (email/recovery) → แลกเป็น session แล้ว redirect (สมัครใหม่ → ตั้งโปรไฟล์, กู้รหัส → `/reset-password`)	token ใช้ได้ครั้งเดียว; เปิดต่างอุปกรณ์ได้
`/auth/callback`	GET	รองรับลิงก์แบบ PKCE (`code`) — ต้องเปิดในเบราว์เซอร์เดียวกับที่เริ่ม จึงไม่ใช้กับอีเมลปัจจุบัน	ต้องมี code verifier cookie
ไฟล์ static (`/images/parks/{640,1280}/*.webp, ฟอนต์, thailand.json`)	GET	รูปอุทยาน ฟอนต์ และเส้นแบ่งจังหวัด — ไม่ผ่าน proxy	แคชด้วย CDN ของ Vercel

4.2 ไคลเอนต์ ↔ Supabase โดยตรง (เบราว์เซอร์ → อินเทอร์เน็ต)

การทำงาน	รายละเอียด
อัปโหลดรูปเข้า Storage	เบราว์เซอร์ลบ EXIF/XMP/GPS ออกก่อน (`strip-metadata.ts`) แล้ว upload ไปที่ `travel-photos/<uid>/record-photos/<id>.ext` หรือ `profile-photos` ด้วย JWT ของผู้ใช้ — policy บังคับให้เขียนได้เฉพาะโฟลเดอร์ของตัวเอง

RPC `save_travel_record`	ส่ง JSON ของบันทึก + รายการรูปที่อัปโหลดแล้ว ฟังก์ชันตรวจว่าไฟล์อยู่ใน Storage ขนาดและชนิดตรงกันจริง ก่อน insert ทั้งหมดในธุรกรรมเดียว; ID ที่ส่งซ้ำ (retry) ไม่สร้างซ้ำ
RPC `save_account_profile`	บันทึกชื่อ/username/bio พร้อม metadata ของอวตาร์/ปก แบบ atomic แล้วคืนรายการไฟล์เก่าที่ต้องลบ
ลบไฟล์เก่า (cleanup)	ลบไฟล์ใน Storage หลังข้อมูลใน DB เปลี่ยนแล้วเท่านั้น (policy ไม่ให้ลบไฟล์ที่ยังถูกอ้างอิง)

4.3 เซิร์ฟเวอร์ ↔ Supabase และบริการอื่น

การเชื่อมต่อ	รายละเอียด
อ่าน catalog `places`	REST ด้วย publishable key (สิทธิ์ SELECT อย่างเดียว) `no-store` แบ่งหน้า 500 แถว timeout 10 วินาที ตรวจรูปแบบข้อมูลก่อนใช้ แคชหนึ่ง snapshot ต่อหนึ่ง request ด้วย React `cache`
อ่านข้อมูลบัญชี	ด้วย SSR client ที่ถือ session ผู้ใช้ (ผ่าน RLS) อ่านเป็นหน้า ๆ ตามจำนวนจริง เพื่อไม่ให้เพดานแถวของโปรเจคตัดข้อมูลผู้ใช้
Supabase Auth → Gmail SMTP	Supabase ส่งอีเมลยืนยัน/กู้รหัสผ่านผ่าน SMTP ที่ตั้งไว้ (เทมเพลตภาษาไทยแก้ที่ Supabase dashboard ไม่ได้ deploy ไปกับ Vercel)

4.4 เบราว์เซอร์ ↔ บริการภายนอก

บริการ	รายละเอียด
Stadia Maps	Leaflet ขอกระเบื้อง PNG จาก `tiles.stadiamaps.com` ส่ง Referer ของเว็บให้ตรวจโดเมน (แคชได้ 24 ชั่วโมง)
นโยบาย CSP	จำกัดแหล่งที่เบราว์เซอร์ติดต่อได้: `img-src 'self' data: blob: tiles.stadiamaps.com`, `connect-src 'self' https://*.supabase.co` , ห้าม frame, ห้าม object

> **สรุปเส้นทางที่ "ไม่" ผ่านเซิร์ฟเวอร์เรา:** ไบต์ของรูปส่วนตัวตอนอัปโหลด (เบราว์เซอร์ → Supabase Storage) และการเรียก RPC ตอนบันทึก ส่วนตอน "ดู" รูป ไบต์ผ่าน `/api/photos` เสมอ เพื่อให้เซิร์ฟเวอร์ตรวจสิทธิ์ก่อนและควบคุมการแคช

5. โครงสร้างไฟล์และหน้าที่ของแต่ละไฟล์

หน้า (pages) และ CSS ส่วนกลางอยู่ใน `app/` ที่ราก ส่วนคอมโพเนนต์ บริการ ข้อมูล ชนิดข้อมูล และ hooks อยู่ใน `src/` (ห้ามสร้าง `src/app/` ซ้ำ)

5.1 รากโปรเจค และ app/ (เส้นทางและ API)

ไฟล์	หน้าที่
`AGENTS.md` / `CLAUDE.md`	ข้อตกลงการทำงานและพฤติกรรมที่ยอมรับแล้วของโปรเจค (แหล่งอ้างอิงหลัก)
`proxy.ts` + `src/lib/supabase/proxy.ts`	รีเฟรช session cookie ก่อนทุกหน้า; ถ่ายทอด cookie/แคช header ไปยัง response
`next.config.ts`	Security headers (CSP, X-Frame-Options, nosniff, Referrer/Permissions policy) และ allowed dev origins
`app/layout.tsx`, `globals.css`	โครงหน้าหลัก ผู้ให้บริการ (AuthProvider, catalog, loading) และ token ดีไซน์/Liquid Glass
`app/page.tsx`	redirect `/` → `/search`
`app/search/page.tsx`	โหลด catalog ฝั่งเซิร์ฟเวอร์ แล้ว render การ์ดสถานที่
`app/places/[id]/page.tsx`	หน้ารายละเอียดสถานที่ (ID ไม่พบ → `notFound()`); รับ `from=passport|profile`
`app/records/new/page.tsx`	ฟอร์มประทับตรา (ต้องล็อกอินจริง)
`app/passport|map|stats|profile/page.tsx`	หน้า Passport / แผนที่ / สถิติ / โปรไฟล์ (ผู้ที่ยังไม่ล็อกอินเห็นหน้า guest)
`app/profile/edit/page.tsx`	แก้ไขโปรไฟล์ และตั้งค่าโปรไฟล์ครั้งแรกหลังสมัคร
`app/login|signup|forgot-password|reset-password/page.tsx`	หน้ากลุ่มยืนยันตัวตน
`app/api/account/route.ts`	GET ข้อมูลบัญชีของผู้ใช้ที่ล็อกอิน
`app/api/photos/[id]/route.ts`	ส่งรูปส่วนตัวพร้อมตรวจสิทธิ์ ย่อด้วย sharp และตั้งแคช
`app/auth/[action]/route.ts`	ตัวจัดการ POST ของ login, signup, resend, forgot-password, logout, ตั้งโปรไฟล์เสร็จ, ลบบัญชี, เปลี่ยนรหัสผ่าน
`app/auth/callback`, `app/auth/confirm`	แลก code/token_hash จากลิงก์อีเมลเป็น session
`app/global-error.tsx`	หน้า error รวม (เช่น อ่านข้อมูลบัญชีครั้งแรกไม่สำเร็จ)

5.2 src/lib (โครงสร้างพื้นฐาน)

ไฟล์	หน้าที่
`lib/supabase/{client,server,route,proxy,config}.ts`	สร้าง Supabase client สำหรับเบราว์เซอร์ / Server Component / Route Handler / proxy และอ่านค่า URL+key จาก env
`lib/auth/server.ts`, `browser.ts`	`getAuthUser` (ยืนยันด้วย getUser), บังคับล็อกอิน (`requireAuthUser`), สร้างแถว profile ที่ขาด

`lib/auth/redirects.ts`, `profile-setup.ts`	อนุญาตเฉพาะปลายทาง redirect ที่ปลอดภัย · กติกาตั้งโปรไฟล์ครั้งแรก (ห้ามใช้ชื่อ/username ตั้งต้น)
`lib/account/server.ts`, `read-rows.ts`	โหลดโปรไฟล์/บันทึก/รูปของบัญชีแบบแบ่งหน้าตามจำนวนจริง
`lib/account/browser.ts`	**Account store** ฝั่งเบราว์เซอร์: snapshot, refresh, อัปโหลดรูป, เรียก RPC, สัญญาณข้ามแท็บผ่าน localStorage
`lib/images/{strip-metadata,image-import}.ts`	ลบ EXIF/GPS, ตรวจชนิด/ขนาด, อ่านไฟล์เป็น data URL
`lib/use-*.ts`, `utils.ts`	hooks: ผู้ใช้ปัจจุบัน, read model ของบันทึก, นำเข้ารูป; ฟังก์ชันเล็ก ๆ

5.3 src/domain, services, data, types (ตรรกะล้วนและข้อมูล)

ไฟล์	หน้าที่
`types/index.ts`	ชนิดข้อมูลหลัก: `Place` (+ `PlaceImage`), `TravelRecord`, Region, PlaceType
`services/place.service.ts`, `place-catalog-source.ts`	โหลด catalog จาก Supabase (`server-only`) พร้อมตรวจชนิดข้อมูลและแบ่งหน้า
`domain/place-catalog.ts`	สร้างดัชนีค้นหา (normalize ข้อความครั้งเดียวต่อ snapshot), ตัวนับอุทยานที่เข้าเกณฑ์
`domain/travel-record-read-model.ts`	รวม catalog + บันทึกของผู้ใช้ → สถิติ รูปรวมต่อสถานที่ ข้อมูลแผนที่ (ไม่คำนวณซ้ำถ้าข้อมูลเดิม)
`domain/account-data.ts`, `place-display.ts`, `place-image.ts`	ตรวจรูปแบบ snapshot บัญชี / ป้ายประเภท ภาค / URL และเครดิตรูปอุทยาน
`data/place-identities.ts`, `archived-places.ts`	ตารางนามแฝงของ ID เก่า (เช่น สามร้อยยอด) และข้อมูลสถานที่ที่อ่านย้อนหลังได้แต่ไม่แสดงในการค้นหา

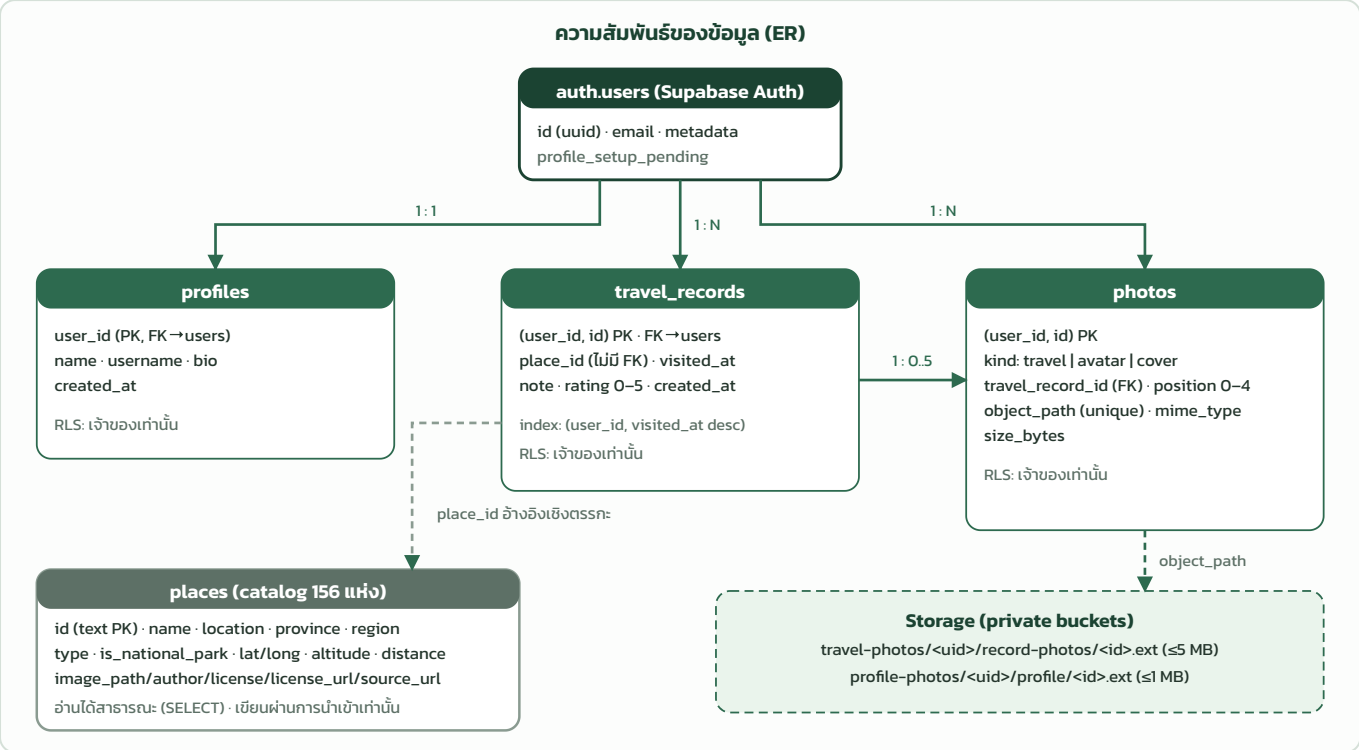
5.4 src/components (UI แยกตามฟีเจอร์)

โฟลเดอร์	หน้าที่ของคอมโพเนนต์หลัก
`navigation/`	TopNav (เดสก์ท็อป), BottomNav (มือถือ), `activeNav.ts` (ลิงก์/แท็บที่ active), NavigationState, RouteLoadingTransition (หน้าโหลดนักเดินป่า)
`search/`	SearchHero, SearchBar, ตัวกรองประเภท/เคยไปแล้ว (แถบ sticky พร้อม pill เลื่อน), `useSearchFilters`, SearchResults, PlaceList, PlaceCard (Server Component), ปุ่ม Stamp, VisitCount
`place/`	CatalogContent/Provider (ส่ง catalog ลง client), PlaceDetailContent, PlaceGallery + PhotoLightbox, PlaceInfo/Chips/Description, PlaceRecord, PlaceActions, BackButton (ใช้ href ชัดเจน ไม่ใช้ history)
`records/`	TravelRecordForm (วันที่ รูป โน้ต ดาว + ตราตัวอย่าง), PhotoUploader
`passport/`, `stats/`, `profile/`	การ์ดบันทึก; ภาพรวมสถิติ วงแหวนความคืบหน้า จังหวัด/ภาค/ประเภท/อุทยาน; ส่วนหัวโปรไฟล์ แกลเลอรีรูป ฟอร์มแก้ไข
`map/`	ThailandMap (Leaflet), DynamicThailandMap (โหลดไม่ใช้ SSR), MapOverview, SelectedPlaceCard, ป้ายชื่อจังหวัด
`auth/`, `ui/`	AuthProvider/Panel/LoginForm/LogoutButton; ส่วนประกอบกลาง: InkStamp (ตราหมึก), HikerLoader, PlaceTypeIcon, StatCard, GuestPrivatePage

5.5 supabase/, data/, scripts/, tests/, docs/

ไฟล์/โฟลเดอร์	หน้าที่
`supabase/migrations/*.sql`	ประวัติโครงสร้างฐานข้อมูล: สร้าง places, ข้อมูลส่วนตัว+RLS, Storage, ลบบัญชีตัวเอง, กฎกู้รหัสผ่าน, คอลัมน์รูปอุทยาน
`supabase/seed.sql`	สร้างจาก CSV โดยสคริปต์ (`on conflict … do update` , รันซ้ำได้)
`supabase/templates/*.html` , `config.toml`	เทมเพลตอีเมลยืนยัน/กู้รหัสผ่าน (ต้องวางใน dashboard ด้วยเอง) และตั้งค่า Supabase ในเครื่อง
`data/national-parks-156.csv`	ต้นฉบับ catalog ที่ผ่านการตรวจ (รวมคอลัมน์รูปและเครดิต 5 คอลัมน์)
`data/place-photos.json`	ผลค้นรูปจาก Commons และบันทึกการคัดทิ้ง (`approved: false`)
`public/images/parks/{640,1280}/`	ไฟล์ WebP รูปอุทยาน 123 แห่ง (รวมราว 34 MB)
`scripts/import-national-parks.py`	ตรวจ CSV → สร้าง `seed.sql` ; `--check` ตรวจว่า seed ล้าสมัยหรือไม่; `--json` ให้ข้อมูลกับเทสต์
`scripts/find|build-place-photos.mjs` , `merge-place-photos.py`	ค้นรูป → ดาวน์โหลดและย่อ → เติมคอลัมน์ลง CSV
`scripts/scan-secrets.mjs` , `test.mjs` , `setup-auth-test-users.mjs`	ตรวจความลับหลุดใน git · ตัวรัน test · สร้างบัญชีทดสอบ QA ผ่าน Admin API
`tests/*.test.ts` , `tests/e2e/*.spec.ts`	unit/DB test (PGlite: migration, seed, RLS, backup/restore) และ e2e Playwright
`.github/workflows/ci.yml`	ขั้นตอน CI
`docs/*.md`	คู่มือเชิงลึก: places-backend, private-data-backend, supabase-setup, security, testing ฯลฯ

6. โมเดลข้อมูลและความสัมพันธ์ของข้อมูล



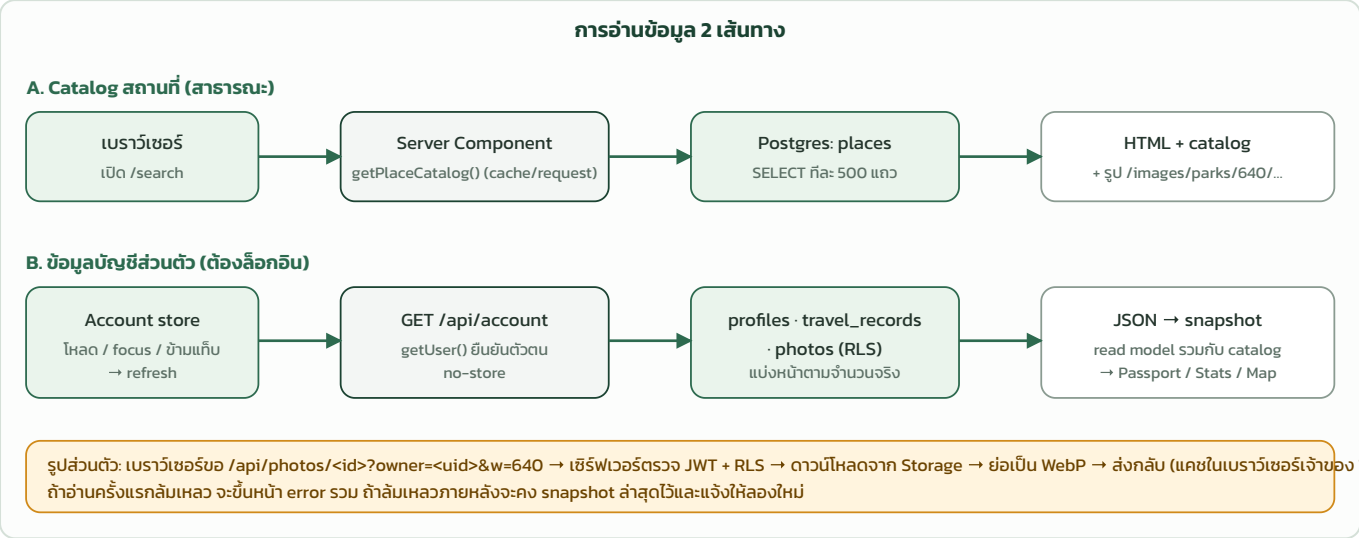
รูปที่ 2 ความสัมพันธ์ของข้อมูล เส้นทึบ = foreign key เส้นประสีเทา = อ้างอิงเชิงตรรกะ (ไม่มี FK) เส้นประสีเขียว = ชี้ไปยังไฟล์ใน Storage

จุดออกแบบที่ควรรู้

- `travel_records.place_id` **ไม่มี foreign key ไป** `places` **โดยตั้งใจ** เพื่อให้บันทึกเก่าที่อ้าง ID ซึ่งเปลี่ยนชื่อหรือถูกเลิกใช้ ยังอ่านได้ ไม่ทำให้ catalog ใหม่ทำลายข้อมูลผู้ใช้ มีตารางนามแฝงใน `place-identities.ts` และข้อมูลสถานที่ย้อนหลังใน `archived-places.ts`
- **ID ของระเบียนผู้ใช้เป็นข้อความที่ผูกกับบัญชี** (`(user_id, id)` เป็น primary key) ส่ง ID ซ้ำหมายถึง retry จึงไม่สร้างบันทึกซ้ำ
- **รูปต่อบันทึกจำกัด 5 รูป** บังคับที่ฐานข้อมูล (`position` 0–4 และ unique index) ส่วนสถานที่หนึ่งแห่งแสดงรูปรวมจากหลายบันทึกได้ไม่จำกัด
- **รูปอุทยาน (catalog) ไม่อยู่ใน Storage** เก็บเป็นไฟล์ static ใน `public/images/parks/` โดย DB เก็บเฉพาะชื่อไฟล์และเครดิต
- ***สถิติไม่ได้เก็บในฐานข้อมูล*** คำนวณทุกครั้งจากบันทึกของผู้ใช้ + catalog (นับสถานที่/จังหวัด/อุทยานที่ไม่ซ้ำ)
- ชนิด `Place.type` มี 4 ค่า (mountain, waterfall, cave, island) ส่วนการนับ "อุทยานแห่งชาติ" ใช้ฟิลด์ `is_national_park` แยกต่างหาก ไม่ขึ้นกับประเภทที่แสดง

7. Data Flow

7.1 การอ่านข้อมูล (Read path)



รูปที่ 3 เส้นทางการอ่าน: catalog สาธารณะ (A) และข้อมูลส่วนตัว (B)

7.2 การเขียนข้อมูล (Write path) — บันทึกการเดินทาง

1. ผู้ใช้เลือกรูป (JPG/PNG/WebP ≤5 MB, ≤5 รูป) → เบราว์เซอร์ **ลบ EXIF/GPS** และแสดงตัวอย่าง
2. เมื่อกดบันทึก account store **ยืนยันตัวตนปัจจุบัน** แล้วอัปโหลดรูปเข้า `travel-photos/<uid>/…` โดยตรง
3. เรียก RPC `save_travel_record(record, photos)` — ฐานข้อมูลตรวจว่าไฟล์อยู่จริง ขนาด/ชนิดตรง แล้ว insert บันทึก + แถวรูปในธุรกรรมเดียว (ID เดิม = no-op)
4. สำเร็จ: ส่งสัญญาณข้ามแท็บ (localStorage) → รีเฟรช snapshot → ไปหน้า Passport
5. ล้มเหลว: ฟอร์มยังเก็บร่างไว้; ระบบพยายามลบไฟล์ที่อัปโหลดไปแล้วแต่ไม่ได้ผูก (ถ้าลบไม่ได้อาจเหลือไฟล์ค้างส่วนตัว ซึ่งต้องล้างภายหลัง)

7.3 การไหลของข้อมูล catalog (ก่อนถึงผู้ใช้)

```
CSV (data/national-parks-156.csv)        ← แก้ที่นี่ที่เดียว (รวมคอลัมน์รูป/เครดิต)
   │  pnpm import:places  (Python: ตรวจ → สร้าง SQL)
   ▼
supabase/seed.sql                          ← สร้างอัตโนมัติ ห้ามแก้มือ
   │  pnpm db:seed:places  (supabase db query --linked)
   ▼
public.places (Postgres)  ──REST/SELECT──►  Server Component  ──►  HTML ที่ผู้ใช้เห็น
public/images/parks/*.webp ──────────── (ไฟล์ static ผ่าน CDN)  ──►  การ์ด/หน้ารายละเอียด
```

8. Workflow

8.1 ขั้นตอนพัฒนาและส่งมอบ

1. แก้โค้ดในเครื่อง (`pnpm dev` ; ถ้ามี dev server ของผู้ใช้อยู่แล้วให้ใช้ตัวเดิม) — dev server ใช้ฐานข้อมูล Supabase ชุดเดียวกับ production
2. ตรวจ: `pnpm exec tsc --noEmit` , `pnpm lint` , `git diff --check` , `pnpm test` (ต้อง Node 22+), `pnpm build` เมื่อจำเป็น
3. ตรวจหน้าจริงบนเดสก์ท็อปและมือถือเมื่อแก้ UI (อ่านโค้ดหรือ lint อย่างเดียวไม่นับเป็นการตรวจภาพ)
4. commit แยกตามประเภท `feat:` `fix:` `style:` `copy:` `docs:` และอัปเดต `AGENTS.md` เมื่อพฤติกรรมหรือการไหลของข้อมูลเปลี่ยน
5. `git push` → GitHub Actions CI → Vercel deploy จาก `main`

8.2 CI (`.github/workflows/ci.yml`)

#	ขั้นตอน	ทำอะไร
1	Secret scan	ตรวจว่าไม่มีรหัส/คีย์หลุดเข้า git
2	Dependency audit (production, high+)	ล้ม CI ถ้ามีช่องโหว่ระดับ high ขึ้นไปใน dependency ที่ใช้จริง (เคยจับช่องโหว่ Next.js ได้เมื่อ 23 ต.ค.)
3	Dependency audit (ทั้งหมด)	แจ้งเตือนอย่างเดียว (ช่องโหว่ dev-only เช่น `braces` ยังเปิดอยู่)
4–5	`tsc --noEmit` , `lint`	ตรวจชนิดข้อมูลและกฎโค้ด
6	Unit/migration/RLS/backup-restore	รัน migration และ seed จริงบน PGlite, ตรวจสิทธิ์ RLS, ทดสอบสำรอง/กู้คืน
7	`pnpm build`	build production

หมายเหตุ: การ push หลาย commit รวดเดียวทำให้ CI ทำงานทีละ commit ถ้า commit กลางทางไม่สมบูรณ์ (เช่น ลบไฟล์แต่ยังมี import ค้าง) CI ของ commit นั้นจะแดงชั่วคราว (เคยเกิดขึ้นเมื่อ 25 ต.ค.)

8.3 การเปลี่ยนโครงสร้างฐานข้อมูล

1. เขียนไฟล์ migration ใหม่ใน `supabase/migrations/` พร้อมเทสต์ PGlite
2. `supabase db push --linked --dry-run` ตรวจรายการที่จะถูก apply แล้วจึง `db push --linked`
3. ถ้าข้อมูลมาจาก CSV: `pnpm import:places` → `pnpm db:seed:places`
4. **apply migration ก่อน แล้วค่อย deploy โค้ดที่อ่านคอลัมน์ใหม่** ไม่เช่นนั้นการโหลด catalog จะล้ม (`column … does not exist`)

8.4 การเพิ่ม/อัปเดตรูปอุทยาน

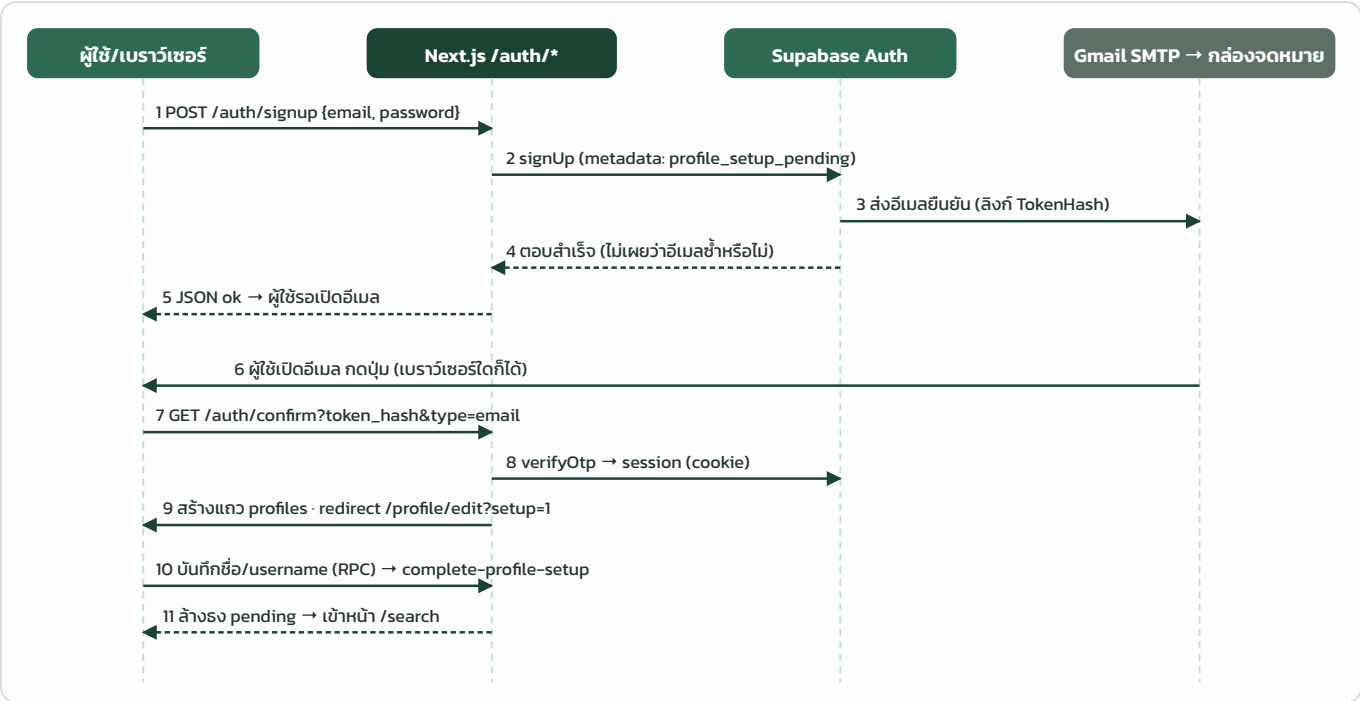
```
node scripts/find-place-photos.mjs      # ค้นผู้สมัครจาก Commons → data/place-photos.json
(ตรวจด้วยตา ตั้ง "approved": false กับรูปที่ไม่เหมาะ)
node scripts/build-place-photos.mjs     # ดาวน์โหลด + ย่อ 640/1280 → public/images/parks/
python3 scripts/merge-place-photos.py   # เติมคอลัมน์ image_* ลง CSV
pnpm import:places && pnpm db:seed:places
```

8.5 การตั้งค่าอีเมล (ทำที่ dashboard)

แก้เทมเพลตใน Supabase → Authentication → Emails ทุกครั้งที่ไฟล์ใน `supabase/templates/` เปลี่ยน (Vercel ไม่ deploy ให้) ลิงก์ในอีเมลต้องเป็นแบบ `{{ .SiteURL }}/auth/confirm?token_hash={{ .TokenHash }}&type=email|recovery` และ Site URL ต้องเป็น `https://doen-pa.vercel.app`

9. สถานการณ์ใช้งาน (Scenarios)

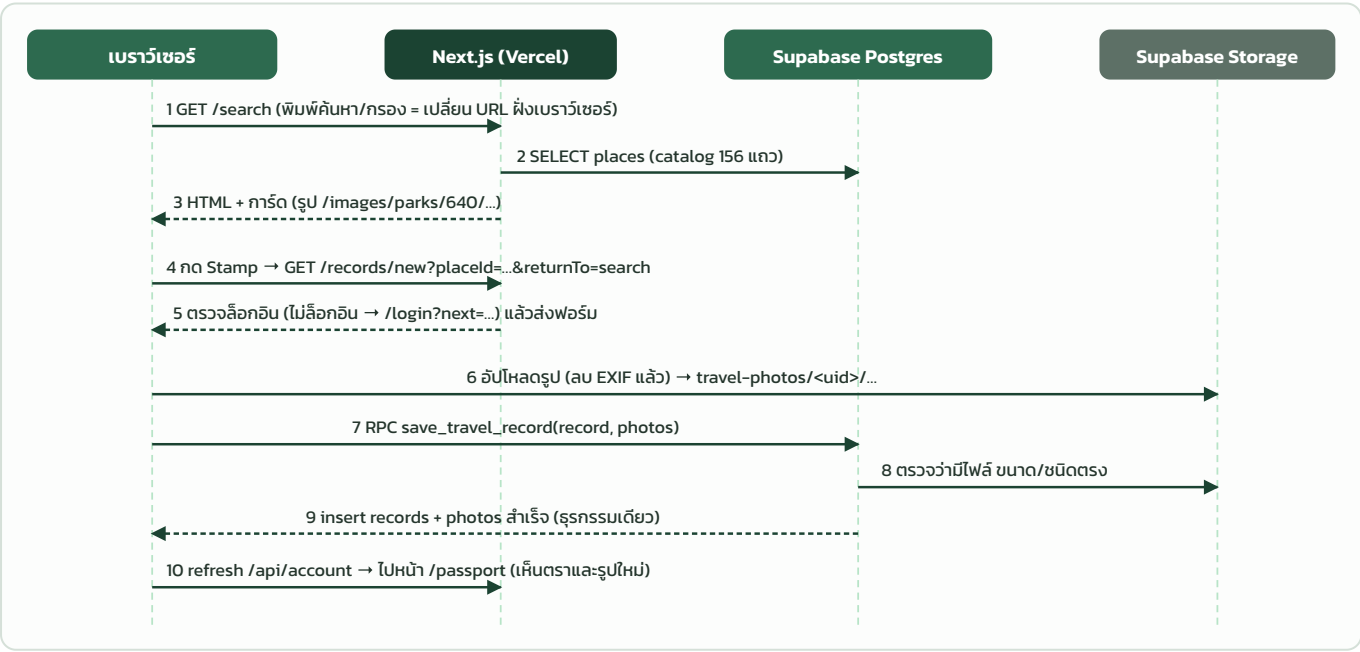
สถานการณ์ A: ผู้ใช้ใหม่สมัครและตั้งโปรไฟล์



รูปที่ 4 ลำดับการสมัครและยืนยันอีเมล

กรณีพิเศษ: (1) สมัครด้วยอีเมลที่มีบัญชีแล้ว Supabase ตอบสำเร็จแต่ **ไม่ส่งอีเมล** (กันการเดาอีเมล) ผู้ใช้ควรไปหน้าเข้าสู่ระบบ (2) กดส่งซ้ำเร็วกว่า 60 วินาที หรือเกินโควต้าอีเมลต่อชั่วโมง จะได้ข้อความ "ทำรายการบ่อยเกินไป" (3) อีเมลองค์กร/มหาวิทยาลัยอาจกัก/ทิ้งอีเมลเงียบ ๆ (ส่งออกแล้วแต่ไม่ถึง) (4) ปุ่มส่งซ้ำมีตัวนับถอยหลัง 60 วินาที

สถานการณ์ B: ค้นหาอุทยานแล้วประทับตรา



รูปที่ 5 ลำดับการค้นหาและประทับตรา

สถานการณ์ C: ดูรูปและแผนที่ที่เคยไป

- **หน้ารายละเอียดจาก Search:** ถ้ายังไม่มีรูปของตัวเอง เห็นรูปอุทยานพร้อมเครดิต (ข้อความธรรมดา); เมื่อมีรูปของตัวเอง รูปของผู้ใช้จะแทนที่ (รูปส่วนตัวผ่าน `/api/photos`)
- **หน้ารายละเอียดจาก Passport/Profile:** โหมดอ่านอย่างเดียว ไม่มีปุ่ม Stamp การ์ดข้อมูลกว้างเต็มคอลัมน์
- **แผนที่:** แสดงหมุดเฉพาะสถานที่ที่เคยไปและมีพิกัด; เลือกหมุดแล้วขึ้นการ์ดข้อมูล (ไม่นำทางไปไหน) กระเบื้องมาจาก Stadia โดยตรง

สถานการณ์ D: ลืมรหัสผ่าน

กรอกอีเมลที่ `/forgot-password` → Supabase ส่งลิงก์ (`type=recovery`) → `/auth/confirm` แลก session แล้วพาไป `/reset-password` เสมอ → ตั้งรหัสใหม่ → ระบบออกจากระบบและพาไปหน้า Login ให้เข้าด้วยรหัสใหม่ (บัญชีที่ยังไม่ยืนยันอีเมลไม่สามารถกู้รหัสได้)

สถานการณ์ E: ผู้ใช้ออกจากระบบ/ลบบัญชี

ออกจากระบบ: ยกเลิก session ปัจจุบัน (`scope: local`) ล้าง cookie และแคชรูป แจ้งแท็บอื่น (ไม่ออกจากอุปกรณ์อื่น) · ลบบัญชี: endpoint `/auth/delete-account` ตรวจรหัสผ่านซ้ำ → ลบแถวรูปและไฟล์ใน Storage → `delete_my_account()` ลบผู้ใช้ (ข้อมูลที่เกี่ยวข้องหายตาม cascade) **ปัจจุบันยังไม่มีปุ่มใน UI ที่เรียก endpoint นี้** (ถูกเอาออกตามคำขอ)

10. ความปลอดภัยและความเป็นส่วนตัว

ด้าน	มาตรการในระบบ
การยืนยันตัวตน	Supabase Auth; proxy รีเฟรชด้วย `getClaims` (ตรวจ JWT ในเครื่อง) ส่วนก่อน render ข้อมูลส่วนตัวใช้ `getUser` ยืนยันกับเซิร์ฟเวอร์ Auth; cookie SameSite=Lax
การควบคุมสิทธิ์	RLS แบบ "เจ้าของเท่านั้น" ทุกตารางส่วนตัวและ bucket ส่วนตัว; catalog ให้ SELECT อย่างเดียวแก่ anon/authenticated ไม่มีสิทธิ์เขียน/ลบ/truncate; การนำเข้าใช้สิทธิ์ผู้ดูแลนอกโค้ดไคลเอนต์
ข้อมูลใน request	รับเฉพาะคำขอ JSON same-origin ที่ `/auth/*`; รายการปลายทาง redirect (`next`) ผ่านรายการที่อนุญาตเท่านั้น (กัน open redirect)
ความเป็นส่วนตัวของรูป	ลบ EXIF/XMP/GPS ในเบราว์เซอร์ก่อนอัปโหลด; รูปส่วนตัวไม่เป็นสาธารณะ ส่งผ่านเซิร์ฟเวอร์ที่ตรวจสิทธิ์; แคชเฉพาะเบราว์เซอร์เจ้าของและล้างด้วย `Clear-Site-Data` เมื่อ login/logout; ไม่ใช้ตัวปรับรูปของ Next (ไม่ให้ไบต์ส่วนตัวเข้าแคชร่วม)
Header ความปลอดภัย	CSP จำกัดแหล่งที่มา, X-Frame-Options DENY, nosniff, Referrer-Policy, Permissions-Policy (ปิดกล้อง ไมค์ ตำแหน่ง การชำระเงิน)
ความลับ	App Password ของ Gmail อยู่ใน Supabase เท่านั้น; คีย์ผู้ดูแลไม่อยู่ใน git/แอป; CI สแกนความลับ
ห่วงโซ่อุปทาน	CI ตรวจช่องโหว่ dependency ระดับ high ขึ้นไป (production)

ข้อจำกัดที่เอกสารของโปรเจคยอมรับไว้: ไม่มี rate limit ฝั่งแอปสำหรับ login (พึ่ง Supabase) · CSP ยังต้องใช้ `'unsafe-inline'` สำหรับสคริปต์ (ข้อมูล hydration ของ Next) · เมื่อตรวจ CSP แล้ว flow ที่ล็อกอิน (อัปโหลด แก้โปรไฟล์) ยังไม่ได้ตรวจใต้ CSP ควรรัน production journey หลัง deploy

11. สิ่งที่ควรรู้ ข้อจำกัด และความเสี่ยง

11.1 เรื่องที่ทำให้ทำงานผิดพลาดได้ง่าย

- **Next.js 16 ต่างจากที่เคยรู้จัก** อ่านคู่มือใน `node_modules/next/dist/docs/` ก่อนเขียนโค้ด (ใช้ `proxy.ts`, build ด้วย Webpack)
- **Node 22+** สำหรับเทสต์ ถ้าใช้ Node 20 ที่เป็นค่าเริ่มต้นบางเครื่อง เทสต์ 5 ข้อล้มด้วย "native WebSocket not found" (ไม่ใช่บั๊กโค้ด) CI เองยังตั้ง Node 20 และมีคำเตือน deprecated
- **dev server ใช้ฐานข้อมูลจริง** `.env.local` ชี้ไป Supabase production ไม่มีฐานข้อมูล staging ดังนั้นการทดสอบฝั่งเขียนในเครื่องเขียนข้อมูลจริง (ใช้บัญชี QA ที่แยกไว้)
- **ลำดับ deploy** migration ก่อน แล้วค่อยโค้ดที่อ่านคอลัมน์ใหม่ (บทที่ 8.3)
- **เทมเพลตอีเมลต้องวางเองใน dashboard** การ push โค้ดไม่เปลี่ยนค่าบน Supabase
- **ห้ามนำข้อมูลตัวอย่างกลับมา** ไม่ใส่บันทึกหรือรูปตัวอย่างของผู้ใช้; ห้ามใช้ URL รูป Stitch (`lh3.googleusercontent.com/aida-public/…`) เป็นแหล่งรูปจริง
- **alt text ของรูป** กรอกได้ในฟอร์มแต่ไม่ถูกบันทึก (ต้องตัดสินใจเรื่องโมเดลข้อมูลก่อนแก้)

11.2 บริการภายนอกและข้อจำกัดของแผนที่ใช้อยู่

เรื่อง	สถานะ/ความเสี่ยง
Stadia Maps	ช่วงทดลอง Professional 14 วันจะหมดราว 5 พ.ย. 2569 แล้วย้ายไปแผน **Free** (200,000 เครดิต/เดือน, **ห้ามใช้เชิงพาณิชย์**) โปรเจคส่วนตัวไม่มีรายได้จึงใช้ได้ แต่ถ้าเกินโควต้าแผนที่อาจโหลดไม่ได้ และถ้าแอปกลายเป็นเชิงพาณิชย์ต้องเปลี่ยนแผนหรือผู้ให้บริการ ควรเปิดหน้าแผนที่บนเว็บจริงหลังวันที่หมดทดลองเพื่อยืนยัน
การส่งอีเมล	ใช้ Gmail SMTP ของบัญชีผู้ส่งธรรมดา: โอกาสเข้าสแปม/ถูกกรอง (โดยเฉพาะอีเมลสถาบัน เช่น `@go.buu.ac.th`) ค่า rate limit อีเมล 30 ฉบับ/ชม. ทั้งโปรเจค และเว้น 60 วินาทีต่อผู้ใช้ (ตามที่ผู้ดูแลแจ้ง) และข้อจำกัดอีเมล Gmail ต่อวัน — ทางแก้ระยะยาวคือใช้โดเมนของตัวเองกับผู้ให้บริการอีเมลธุรกรรม พร้อม SPF/DKIM/DMARC
รูปอุทยานจาก Commons	แสดงเครดิตเป็นข้อความธรรมดา (ผู้ถ่าย · สัญญา) ซึ่งครบข้อบังคับหลักของ CC BY/BY-SA แต่ CC BY 4.0 ยังขอลิงก์ไปยังสัญญา "เมื่อทำได้สมควร" ลิงก์ต้นทางและสัญญายังเก็บใน DB พร้อมทำหน้ารวมเครดิตได้ · 33 อุทยานยังไม่มีรูป (แสดงกรอบ topo)
แผน Supabase Free	ควรตรวจนโยบายสำรองข้อมูลและโควต้า (ฐานข้อมูล/Storage/แบนด์วิธ) ของแผนที่ใช้อยู่ ว่ารองรับการเติบโตและการกู้คืนที่ต้องการหรือไม่ (รายงานนี้ไม่ได้ตรวจค่าตั้งของแผน)

11.3 ประสิทธิภาพ

- หน้า Search ฝัง catalog 156 แห่งมาในหน้าเดียวแล้วกรองในเบราว์เซอร์ จึงตอบสนองเร็วหลังโหลดแล้ว แต่การโหลดแรกใหญ่ขึ้นเมื่อ catalog โตขึ้นหรือมีรูปมาก
- หน้าโหลด "นักเดินป่า" รอรูปบนหน้าจอให้โหลดเสร็จ (สูงสุด 10 วินาที) บน dev server เห็นหน้า Search ครั้งแรกช้า 10–15 วินาที ซึ่งยังไม่ได้วัดบน production
- รูปอุทยาน 123 รูปหนักราว 34 MB ใน git (ประวัติไบนารีสะสม) ควรระวังการเปลี่ยนรูปบ่อย ๆ

12. คำแนะนำและสิ่งที่ควรทำต่อ

12.1 ลำดับความสำคัญสูง (ก่อนเปิดให้คนทั่วไปใช้)

#	เรื่อง	เหตุผล / ขั้นตอนโดยสรุป
1	ย้ายการส่งอีเมลไปโดเมนของตัวเอง	แก้ปัญหาอีเมลเข้าสแปม/ถูกอีเมลสถาบันกรอง ใช้ผู้ให้บริการอีเมลธุรกรรม + SPF/DKIM/DMARC แล้วใส่ใน Custom SMTP ของ Supabase
2	ยืนยันแผนที่หลังช่วงทดลอง Stadia	ตรวจหลัง 5 พ.ย.; ตั้งแจ้งเตือนการใช้งาน; เตรียมแผนสำรองถ้าเกินโควต้าหรือเปลี่ยนเป็นเชิงพาณิชย์
3	ปรับ rate limit อีเมลและเพิ่มข้อความช่วยผู้ใช้	เพิ่มโควต้าต่อชั่วโมงให้เหมาะกับจำนวนผู้ใช้; แจ้งในหน้าสมัครให้ตรวจโฟลเดอร์สแปม; แจ้ง "อีเมลนี้มีบัญชีแล้ว" อย่างระมัดระวังเรื่องความเป็นส่วนตัว (ต้องตัดสินใจเชิงนโยบาย)
4	ตรวจ flow ที่ล็อกอินใต้ CSP แล้วรัน production journey	เอกสารความปลอดภัยระบุว่ายังไม่ได้ตรวจการอัปโหลดรูปและแก้โปรไฟล์ภายใต้ CSP
5	ตั้ง CI และเครื่องพัฒนาให้ใช้ Node 22	ตัดคำเตือน Node 20 deprecated; ใส่ `.nvmrc` ; เทสต์ในเครื่องจะผ่านครบ
6	ตั้งการแจ้งเตือนค่าใช้จ่าย/การใช้งาน	Vercel และ Supabase ตามที่เอกสารความปลอดภัยแนะนำไว้

12.2 ระยะกลาง

- **แยกฐานข้อมูล staging** หรือ Supabase branch เพื่อไม่ให้ dev server เขียนข้อมูล production
- **หน้า "เครดิตรูปภาพ"** รวมทั้ง 123 รูปพร้อมลิงก์ต้นทาง/สัญญา เพื่อให้ครบตามเงื่อนไข CC BY โดยไม่ทำให้การ์ดรก
- **หารูปให้ 33 อุทยานที่ยังไม่มี** (ถ่ายเอง ขออนุญาตกรมอุทยานฯ หรือค้นเพิ่มจาก Commons ด้วยคำค้นอื่น) และตัดสินใจนโยบายรูปสถานที่ถาวร
- **บันทึก alt text ของรูป** (เพิ่มคอลัมน์ใน `photos` และ RPC) เพื่อความเข้าถึงได้
- **วัดประสิทธิภาพบน production build** (หน้า Search ครั้งแรก, ขนาดหน้า) แล้วพิจารณาแบ่งหน้า/โหลดรูปตามหน้าจอ
- **Nonce สำหรับ CSP** เพื่อเลิกใช้ `'unsafe-inline'` (ต้องเทียบกับต้นทุนที่ทุกหน้าต้อง render ตามคำขอ)
- **วางแผนสำรองข้อมูล** ตรวจ/ซ้อมกู้คืนข้อมูลผู้ใช้ให้เหมาะกับแผน Supabase ที่ใช้

12.3 ฟีเจอร์ที่เลื่อนไว้ (ต้องได้รับอนุมัติก่อน)

Google login · หน้าสะสม badge · (นอกขอบเขต MVP: social, รีวิวสาธารณะ ฯลฯ) — ก่อนทำฟีเจอร์ใหม่ ให้ตรวจเป้าหมายผู้ใช้ เส้นทาง คอมโพเนนต์ data flow การแสดงผลบนมือถือ และการเข้าถึง และถามเมื่อเรื่องผลิตภัณฑ์/ดีไซน์/ข้อมูลยังไม่ชัดเจน

12.4 ข้อแนะนำการทำงานกับโค้ดนี้

- ใช้ `AGENTS.md` เป็นแหล่งความจริงของพฤติกรรมที่ยอมรับแล้ว และอัปเดตพร้อมโค้ดเสมอ
- แยก commit ตามประเภท และให้แต่ละ commit รัน `tsc` / `lint` ผ่าน (กันการแดงชั่วคราวใน CI)
- เก็บการเขียนข้อมูลไว้ใน services/hooks ไม่ใส่ในคอมโพเนนต์แสดงผล และตรวจ loading/empty/error/focus/disabled ทุกครั้งที่แก้การโต้ตอบ
- ห้ามเปลี่ยนแอปให้กลายเป็นตัววางแผน/จอง/โซเชียลโดยไม่ได้รับการอนุมัติ

13. ภาคผนวก

13.1 คำสั่งที่ใช้บ่อย

คำสั่ง	ใช้ทำอะไร
`pnpm dev`	รัน dev server (ถ้ายังไม่มีตัวที่รันอยู่)
`pnpm exec tsc --noEmit` · `pnpm lint`	ตรวจชนิดข้อมูล / กฎโค้ด (ไม่มีสคริปต์ `type-check`)
`pnpm test`	unit + migration/seed/RLS บน PGlite (ใช้ Node 22+)
`pnpm test:e2e`	Playwright e2e (ต้องมีไฟล์บัญชี QA ในเครื่อง)
`pnpm build`	build production (Webpack)
`pnpm import:places` · `pnpm import:places:check`	สร้าง/ตรวจ `supabase/seed.sql` จาก CSV
`pnpm db:seed:places`	รัน seed กับโปรเจค Supabase ที่ลิงก์ไว้
`pnpm exec supabase db push --linked --dry-run`	ดูรายการ migration ที่จะถูก apply (แล้วค่อยรันไม่มี `--dry-run`)
`pnpm audit --prod --audit-level=high`	ตรวจช่องโหว่ dependency แบบเดียวกับ CI

13.2 ตัวแปรสภาพแวดล้อมสำคัญ

ตัวแปร	ความหมาย
`NEXT_PUBLIC_SUPABASE_URL`	URL ของโปรเจค Supabase (สาธารณะ)
`NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`	คีย์สาธารณะ (สิทธิ์ตามนโยบาย RLS)
`NEXT_PUBLIC_STADIA_MAPS_API_KEY` (ไม่บังคับ)	ใช้เมื่อ deploy บนโดเมนที่ยังไม่ได้ยืนยันกับ Stadia

คีย์ผู้ดูแล (service role) ไม่อยู่ในไฟล์ใด ๆ ของแอป สคริปต์ที่ต้องใช้จะอ่านจาก Supabase CLI ที่ล็อกอินไว้ในหน่วยความจำเท่านั้น

13.3 เอกสารอ้างอิงใน repo

ไฟล์	เนื้อหา
`AGENTS.md`	พฤติกรรมและข้อตกลงที่ยอมรับแล้วทั้งหมด
`docs/places-backend.md`	catalog ใน Supabase, การนำเข้า, ลำดับ apply
`docs/private-data-backend.md`	ตารางส่วนตัว ธุรกรรม Storage การ cleanup
`docs/supabase-setup.md`	Auth, อีเมล, redirect, บัญชี QA
`docs/security.md`	ผลรีวิวความปลอดภัยและข้อยอมรับ
`docs/testing.md`	วิธีและผลการทดสอบ production acceptance
`data/README.md`	catalog และขั้นตอนรูปอุทยาน

รายงานนี้จัดทำจากการอ่านโค้ดและเอกสารใน repository และข้อมูลที่ผู้ดูแลโปรเจคแจ้งระหว่างการทำงาน ณ 30 ต.ค. 2569 ค่าตั้งภายนอก (Supabase dashboard, Stadia, Vercel) อาจเปลี่ยนแปลงได้โดยไม่สะท้อนในโค้ด